

Monitoreo IoT con análisis predictivo

Trabajo Práctico Integrador — Bases de Datos para IA (CEIA)

Mariano Jauffroy y Andrés García

Agosto 2026

Índice

1. Descripción del trabajo	3
2. Solución propuesta	3
3. Estructura de carpetas	3
4. Cómo ejecutar todo	4
Informe detallado por actividad	5
1. Análisis del caso de uso	6
1.1 Descripción general	6
1.2 Problema que busca resolver la solución	6
1.3 Usuarios principales	6
1.4 Procesos y funcionalidades que debería soportar	6
1.5 Información que necesita gestionar	7
1.6 Riesgos que aparecen en relación con los datos	7
1.7 Decisiones de diseño importantes para el caso	7
2. Relevamiento de datos necesarios	9
2.1 Introducción	9
2.2 Clasificación de los datos	9
2.3 Ejemplos representativos	9
3. Modelo conceptual	12
3.1 Descripción general	12
3.2 Entidades y atributos relevantes	12
3.3 Relaciones y cardinalidades	12
3.4 Restricciones y decisiones de diseño relevantes	13
4. Modelo lógico relacional	14
4.1 Descripción general	14
4.2 Capa bronce	15
4.3 Capa plata	16
4.4 Capa oro	22
4.5 Diagrama	27
5. Normalización, desnormalización y decisiones de diseño	29
5.1 Criterio general	29
5.2 Normalización en la capa plata	29

5.3 Normalización y desnormalizado en la capa oro	30
5.4 Excepciones de Desnormalizaciones en plata	30
5.5 Resolución de las relaciones N:M	31
5.6 Correspondencia con los patrones de consulta	31
6. Selección tecnológica	32
6.1 Descripción general	32
6.2 Requisitos técnicos que impone el caso	32
6.3 La disyuntiva de fondo: motor único vs. persistencia poliglota	33
6.4 La opción elegida: PostgreSQL + pgvector	33
6.5 Alternativas evaluadas y descartadas	34
6.6 Relación con el contenido de la materia	35
6.7 Conclusión	35
7. Modelo físico e implementación	36
7.1 Descripción general	36
7.2 Organización de los scripts	36
7.3 Orden de creación y dependencias	36
7.4 Particionado de medición	36
7.5 Restricciones e índices	37
7.6 Triggers: detección de eventos y alertas	37
7.7 Carga de datos de ejemplo	38
7.8 Verificación	38
8. Consultas representativas	39
8.1 Descripción general	39
8.2 Las seis consultas	39
8.3 Resultado de la comparación (consulta 6)	39
8.4 Nota sobre el volumen de datos	40
9. Datos semiestructurados, no estructurados y búsqueda vectorial	41
9.1 Descripción general	41
9.2 Datos semiestructurados (JSONB)	41
9.3 Datos no estructurados y búsqueda vectorial	41
9.4 Riesgo de exposición vía IA	41
10. Arquitectura de datos	42
10.1 Descripción general	42
10.2 Diagrama del flujo	42
10.3 Las capas y sus procesos	42
10.4 Consumidores	43
10.5 Límite base / aplicación	43
11. Seguridad, permisos y aislamiento	44
11.1 Descripción general	44
11.2 Doble identidad: roles de base vs usuarios de aplicación	44
11.3 Aislamiento por planta con RLS	44
11.4 Ocultamiento de columnas sensibles	44
11.5 Notas de implementación	45
12. Escalabilidad y rendimiento	46
12.1 El problema de escala	46
12.2 Particionamiento	46
12.3 Índices	46
12.4 Retención diferenciada	46
12.5 Precálculo analítico	47

1. Descripción del trabajo

Este trabajo es una **solución de datos end-to-end** para el monitoreo IoT con análisis predictivo de una planta industrial de dos sitios, con equipos rotativos (motores, bombas, cintas y tableros) instrumentados con sensores. El objetivo de los datos es pasar de un mantenimiento reactivo a uno **predictivo**: a partir de la telemetría de los sensores se detectan eventos anómalos, se generan alertas, se registran las intervenciones de mantenimiento y se sostiene el análisis histórico y predictivo.

El conjunto de datos es la telemetría de esos sensores —del orden de decenas de millones de lecturas al año—, generada sintéticamente a escala realista para validar el diseño, el circuito de detección y las consultas sin depender de una planta real.

2. Solución propuesta

La solución se apoya en una **arquitectura Medallion** sobre un único PostgreSQL (con pgvector), implementada como schemas:

- **Bronce** — ingesta cruda tal como llega (JSONB, sin validar).
- **Plata** — datos limpios y validados, normalizados (3FN), con Row-Level Security para el aislamiento entre plantas.
- **Oro** — modelado final para consumo: agregados y features desnormalizados, más la trazabilidad del modelo predictivo.

Sobre esa base:

- **Analítica**: agregados horarios/diarios y consultas representativas (últimas lecturas, alertas por planta, medias móviles, rankings) que sirven de KPIs y alimentan los tableros de supervisión.
- **Predicción**: el ciclo de ML está modelado en la capa oro (features por ventana, modelo por tipo de equipo, corridas, métricas y predicciones); una predicción que supera el umbral genera una alerta por trigger. El entrenamiento del modelo en sí es externo y queda fuera del alcance.
- **Consumo**: búsqueda vectorial de intervenciones por similitud (pgvector) para el técnico, y una app operativa que consume la capa oro —conceptual, fuera del alcance de este trabajo, ya que la base registra hechos y la aplicación gobierna los procesos con estado.

3. Estructura de carpetas

CEIA_BdD_tp_integrador/	
├─ README.md	Esta guía
├─ docker-compose.yml	Levanta PostgreSQL + pgvector y carga todo
├─ docker/initdb/	Script que auto-carga la base en el primer arranque
├─ db/	Todo el SQL, numerado por orden de ejecución
│ ├─ run_all.sql	Script maestro (arma el modelo base en orden)
│ └─ estructura/	DDL de tablas, triggers (00–08, 11) y seguridad RLS (40)
│ │ └─ indices_vistas/	Particiones e índices (09, 10, 31 vectorial)
│ │ └─ datos/	Carga de datos (12–18) y embeddings (30)
│ │ └─ consultas/	Las 8 consultas representativas (20–26, 32)
└─ docs/	
│ └─ Informe/	Los 12 informes por actividad (.md)
│ └─ Informe_Final.pdf	Informe completo (este documento)

		arquitectura_datos.mmd	Fuente del diagrama de arquitectura
		modelo_conceptual.png	Diagrama del modelo conceptual
		modelo_logico.png	Diagrama del modelo lógico
		anexos/	Contexto del proyecto, guía de implementación y DBML
		nosql/	Notas de datos semiestructurados y vectoriales
		vectorial/	
		data/	Reservada para datos de ejemplo exportados

El informe principal se entrega como PDF (docs/Informe_Final.pdf); todo lo demás —SQL, consultas, diagramas y anexos— queda versionado por separado en las carpetas de arriba.

4. Cómo ejecutar todo

Prerrequisito: Docker Desktop instalado y corriendo.

a) Levantar la base (crea y carga todo solo). Desde la raíz del proyecto:

```
docker compose up -d
```

En el primer arranque, PostgreSQL crea la base tp_bdia y ejecuta automáticamente los scripts de db/ (estructura, particiones, índices, triggers, carga de datos, seguridad RLS y búsqueda vectorial), dejando la base lista sin pasos extra.

b) Verificar que cargó. Datos de conexión: host localhost, puerto 5432, usuario postgres, contraseña bdia, base tp_bdia.

```
docker compose exec db psql -U postgres -d tp_bdia -c "SELECT count(*) FROM plata.medicion;"
```

Debe devolver del orden de un millón de mediciones.

c) Probar las consultas. Están en db/consultas/. Por ejemplo:

```
docker compose exec db psql -U postgres -d tp_bdia -f /db/consultas/25_consulta_6_explain_plata_
```

d) App de consumo. Fuera del alcance de este trabajo: el proyecto entrega la base de datos (donde la aplicación operativa se conectaría a la capa oro), no la aplicación en sí.

Informe detallado por actividad

Las secciones siguientes desarrollan en detalle cada una de las 12 actividades del trabajo.

1. Análisis del caso de uso

1.1 Descripción general

El trabajo trata el caso de **monitoreo IoT con análisis predictivo**. Como impronta propia se plantea el problema sobre industria con equipos rotativos (motores eléctricos, bombas centrífugas, cintas transportadoras y además tableros eléctricos). Adicionalmente, se optó por modelar el escenario con **dos sitios** en lugar de uno solo, lo que obliga a que el diseño contemple desde el inicio el aislamiento de datos entre plantas para ciertos roles.

1.2 Problema que busca resolver la solución

El problema principal que busca resolver la solución es pasar de un esquema de **mantenimiento reactivo a uno predictivo**: hoy las fallas en los equipos rotativos se detectan cuando ya ocurrieron o cuando un operario las nota, y la idea es poder anticiparlas a partir del análisis histórico y en tiempo real de las mediciones de los sensores.

Sobre esa base, la solución termina resolviendo también otros problemas que surgen naturalmente del mismo dominio:

- **Gestión de alarmas**: cuando una medición se sale de rango, el sistema tiene que generar un evento, que podría convertirse en una alerta, y permitir que alguien la atienda.
- **Registro y consulta de intervenciones de mantenimiento**, incluyendo la posibilidad de buscar casos pasados con síntomas similares a partir de las observaciones que deja el técnico, usando búsqueda por similitud sobre texto.
- **Trazabilidad histórica**, para poder analizar tendencias y comparar comportamiento entre dispositivos, líneas o plantas.

1.3 Usuarios principales

Se modelan los siguientes usuarios:

- Operario de línea
- Técnico de mantenimiento
- Supervisor de planta
- Científico de datos
- Administrador o ingeniero de datos

Estos se pueden agrupar según cómo acceden a los datos:

- **Acceso indirecto, a través de una aplicación**: el operario de línea y el técnico de mantenimiento interactúan con el sistema desde apps dedicadas, sin ellos escribir o consultar SQL de forma directa mediante credenciales de base de datos propias.
- **Acceso directo o semidirecto a los datos**: el supervisor de planta y el científico de datos consultan información con mayor libertad, aunque siempre dentro de los límites que impone el control de acceso.
- **Acceso total**: el administrador o ingeniero de datos, responsable de la infraestructura completa.

El detalle de cada rol y su implementación se desarrolla en la sección de seguridad, permisos y aislamiento.

1.4 Procesos y funcionalidades que debería soportar

- Ingesta continua de mediciones desde los sensores de cada dispositivo.
- Validación de esas mediciones antes de que se consideren “confiables” para el resto del sistema.

- Detección automática de eventos anómalos a partir de umbrales.
- Generación y seguimiento de alertas, con asignación a un responsable.
- Registro de órdenes de trabajo e intervenciones de mantenimiento, con texto libre de observaciones.
- Búsqueda de intervenciones pasadas por similitud de síntomas descriptos en lenguaje natural.
- Consulta y análisis histórico de las mediciones.
- Soporte a un componente predictivo que consume los datos históricos para generar predicciones sobre el estado de los equipos.

1.5 Información que necesita gestionar

A grandes rasgos, el sistema tiene que gestionar información de varios tipos: datos de los activos (ubicaciones, dispositivos, sensores y su configuración), datos de telemetría (mediciones, eventos, alertas), datos de gestión (usuarios, órdenes de trabajo, intervenciones) y datos analíticos (features calculadas, modelos entrenados, predicciones). Luego, toda esta información pasa y se distribuye en distintos grados de madurez a lo largo del flujo de los datos (se retoma en detalle en la sección de relevamiento y clasificación de datos).

1.6 Riesgos que aparecen en relación con los datos

Exposición de datos cruzada (aislamiento). Sin un control de acceso adecuado, un usuario podría terminar viendo información que no le corresponde: un supervisor accediendo a datos de una planta que no es la suya, la búsqueda por similitud devolviendo intervenciones de otra planta, o un científico de datos viendo qué técnico específico hizo una intervención (un dato personal que no necesita para su análisis). Este riesgo se encara aplicando Row-Level Security en la capa plata y en la búsqueda vectorial, y combinando RLS con vistas en la capa oro que filtran ciertas columnas sensibles (como la identidad del técnico).

Calidad del dato de sensores. Las lecturas pueden llegar erróneas por un sensor que falla, o fuera de rango, o directamente faltar por pérdida de conectividad, lo que podría generar falsas alarmas o la falta de ellas si no se toman con cuidado. Este riesgo se encara validando el dato entre la capa bronce (crudo, tal como llega) y la capa plata, y haciendo que la detección de eventos corra sobre la capa plata y no sobre el dato crudo. Para los casos de mediciones validadas como fuera de rango o faltantes, también se generan eventos que pueden derivar en órdenes de trabajo, pero apuntando a revisar el sensor, no el dispositivo.

Volumen y retención. Con el volumen estimado para el caso (del orden de 50 a 60 millones de filas por año en mediciones), si hubieran consultas sin optimizar o una retención de datos históricos sin límites, se termina degradando el sistema a medida que crece. Este riesgo se encara acotando la retención del detalle de mediciones a un año en la capa plata (con agregados de más largo plazo en la capa oro), mientras que las órdenes de trabajo e intervenciones (al ser muchos menos registros) se conservan por un período bastante más extenso. Se aplica particionamiento mensual con índices adecuados para ese tipo de dato.

1.7 Decisiones de diseño importantes para el caso

- **Motor único: PostgreSQL.** Se eligió un solo motor, en lugar de combinar varios sistemas especializados, porque cubre series temporales (para los históricos y sistema predictivo), datos semiestructurados (configuración de sensores en JSONB), seguridad por fila y búsqueda vectorial (extensión pgvector) en un solo motor. Más detalles en sección Selección tecnológica.
- **Arquitectura Medallion (Bronce/Plata/Oro).** Los datos se organizan en capas según su nivel de madurez: crudo, validado, analítico. Se implementan como schemas dentro del mismo PostgreSQL. Se desarrolla en la sección de arquitectura de datos.

- **Row-Level Security (RLS).** El aislamiento por planta no queda sólo librado a la lógica de la aplicación, sino que se resuelve también en la base de datos. Se desarrolla en la sección de seguridad, permisos y aislamiento.
- **Mediciones en formato largo/angosto.** Cada lectura de sensor se guarda como una fila, en lugar de una columna por variable, para poder incorporar nuevos sensores o tipos de variable sin tener que rediseñar el modelo. Se desarrolla en la sección de modelo lógico y normalización.
- **Alcance acotado de la búsqueda vectorial.** pgvector se aplica únicamente sobre el texto libre de las observaciones de las intervenciones de los técnicos para encontrar intervenciones pasadas con síntomas parecidos expresados en lenguaje natural. Se desarrolla en la sección de datos semiestructurados, no estructurados y vectoriales.

2. Relevamiento de datos necesarios

2.1 Introducción

En esta sección se retoma la info descripta en 1.5 *Información que necesita gestionar* y se la clasifica según las categorías propuestas por la consigna, con el objetivo de identificar qué tipo de dato es cada uno y qué implicancias tiene sobre su almacenamiento, acceso y tratamiento.

Vale aclarar que estas categorías no son mutuamente excluyentes: un mismo dato puede pertenecer a más de una a la vez. Por ejemplo, el registro de un usuario es a la vez estructurado y sensible; o una alerta abierta es estructurada y operacional. La clasificación simplemente busca asociar los datos a las dimensiones mencionadas en la consigna.

Nota: los ejemplos que se presentan en esta sección son mínimos y preliminares. Su objetivo es ilustrar aproximadamente la naturaleza de cada tipo de dato identificado, y no validar el modelo completo. Las entidades, relaciones, cardinalidades, tablas y consultas se realizarán progresivamente en las secciones correspondientes al modelo lógico, la implementación mínima y las consultas representativas, donde se presentará un conjunto de datos de ejemplo más completo.

2.2 Clasificación de los datos

Tipo	Ejemplos	Capa (Medallion)
Estructurados	dispositivos, sensores, ubicaciones, mediciones, alertas, órdenes de trabajo	plata / oro
Semiestructurados	payload crudo de ingesta (JSONB), configuración de sensor (JSONB: umbrales, calibración)	bronce / plata
No estructurados	observaciones de texto libre cargadas por el técnico en cada intervención	plata
Operacionales	alertas abiertas o en revisión, órdenes de trabajo en curso	plata
Analíticos	features calculadas por ventana, predicciones de falla, métricas de modelos	oro
Sensibles	datos personales de usuarios y técnicos (nombre, email, etc. asociados a una intervención)	plata
Auditoría / trazabilidad	log de cambios de estado de alertas, histórico de configuración de dispositivo	plata

2.3 Ejemplos representativos

a) Datos estructurados

dispositivo

id_dispositivo	nombre	tipo	id_ubicacion
101	Motor-A1	motor_electrico	5

sensor

id_sensor	id_dispositivo	tipo_variable	unidad
501	101	vibracion	mm/s

medicion

id_sensor	timestamp	valor
501	2026-08-10 08:00:00	2.3
501	2026-08-10 08:00:10	2.4
501	2026-08-10 08:00:20	5.8

b) Datos semiestructurados

Payload crudo tal como llega a la capa bronce:

```
{
  "device_id": "101",
  "sensor_id": "501",
  "ts": "2026-08-10T08:00:20Z",
  "vibration_mm_s": 5.8,
  "firmware": "v2.1.3"
}
```

c) Datos no estructurados

Campo de texto libre dentro de intervencion:

intervencion

id_intervencion	id_orden_trabajo	id_tecnico	observaciones
301	201	42	"Se detectó desalineación en el eje del motor. Vibración fuera de rango en dirección radial. Se realizó ajuste y se recomienda seguimiento en 7 días."
302	201	42	"Segunda visita: vibración estabilizada tras ajuste previo. Se cierra intervención."

d) Datos operacionales**alerta**

id_alerta	id_evento	severidad	estado	timestamp_apertura
401	601	alta	abierta	2026-08-10 08:00:20
402	602	media	en_revision	2026-08-10 09:15:00

orden_trabajo (vista operacional, solo estado en curso)

id_orden_trabajo	id_alerta	id_dispositivo	estado
201	401	101	en_curso

e) Datos analíticos

Feature/predicción calculada en la capa oro:

id_dispositivo	ventana	vibracion_media	vibracion_max	prob_falla_7d
101	2026-08-10 (diaria)	3.1	5.8	0.62

f) Datos sensibles**usuario**

id_usuario	nombre	rol	email	id_planta
42	J. Pérez	tecnico_mantenimiento	jperez@planta.com	5

g) Datos de auditoría o trazabilidad**log_cambio_estado_alerta**

id_log	id_alerta	estado_anterior	estado_nuevo	id_usuario	timestamp
701	401	abierta	en_revision	42	2026-08-10 08:45:00

3. Modelo conceptual

3.1 Descripción general

El modelo conceptual representa el dominio de monitoreo IoT con análisis predictivo en cuatro bloques: **Activos** (ubicaciones, dispositivos, sensores y su configuración), **Telemetría** (mediciones, eventos y alertas), **Gestión** (usuarios, órdenes de trabajo e intervenciones) y **Analítico** (modelos de ML, corridas de entrenamiento, métricas y predicciones). El modelo es independiente de la tecnología de implementación; la traducción a tablas relacionales sobre PostgreSQL se desarrolla en la Actividad 4 (modelo lógico).

El diagrama completo se adjunta como docs/modelo_conceptual.png.

3.2 Entidades y atributos relevantes

Bloque	Entidad	Atributos relevantes
Activos	Ubicación	nombre, tipo (planta / área / línea), ubicación padre (autoreferencial)
Activos	Dispositivo	nombre, tipo de dispositivo, ubicación
Activos	Sensor	nombre, tipo de variable, unidad
Activos	Configuración de Dispositivo (historizada)	parámetro, valor, vigente desde / hasta
Telemetría	Medición	timestamp, valor
Telemetría	Evento	timestamp, tipo de evento, descripción
Telemetría	Alerta	severidad, estado, timestamp de apertura
Gestión	Orden de Trabajo	estado, fecha de apertura / cierre
Gestión	Intervención	observaciones (texto libre + embedding), fecha
Gestión	Usuario	nombre, rol, ubicación
Analítico	Modelo (ML)	nombre, tipo, versión
Analítico	Corrida de Entrenamiento	fecha, dataset usado, hiperparámetros
Analítico	Métrica	nombre, valor
Analítico	Predicción	ventana, probabilidad de falla, umbral, timestamp

3.3 Relaciones y cardinalidades

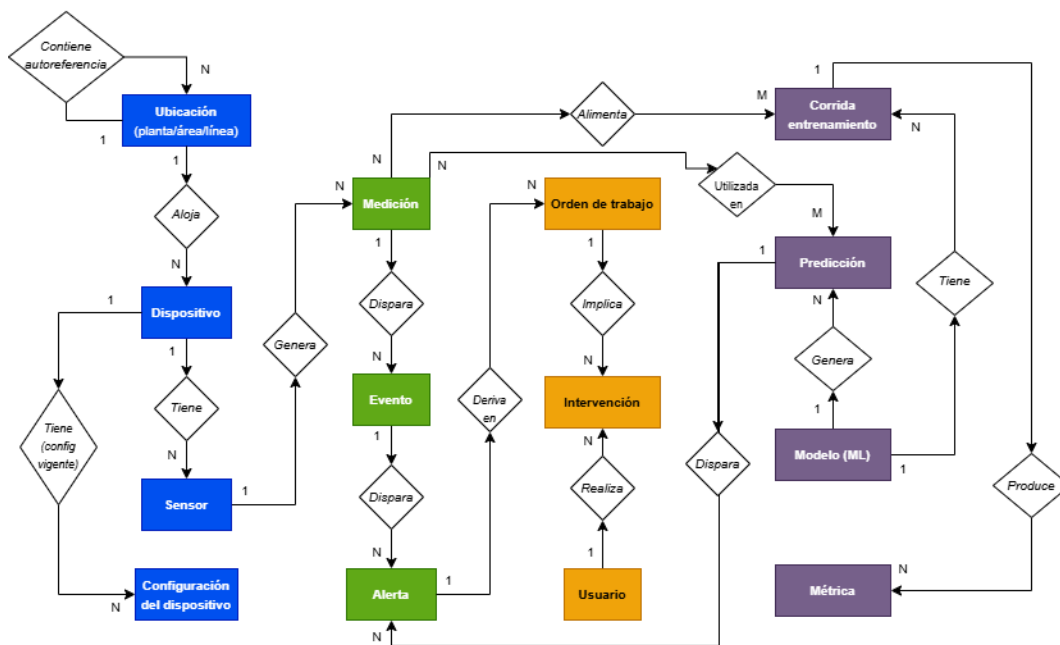
#	Relación	Cardinalidad
1	Ubicación contiene Ubicación	1:N (autoreferencial)
2	Ubicación aloja Dispositivo	1:N
3	Dispositivo tiene Sensor	1:N
4	Dispositivo tiene (config. vigente) Configuración de Dispositivo	1:N

#	Relación	Cardinalidad
5	Sensor genera Medición	1:N
6	Medición dispara Evento	1:N
7	Evento dispara Alerta	1:N
8	Alerta deriva en Orden de Trabajo	1:N
9	Orden de Trabajo implica Intervención	1:N
10	Usuario realiza Intervención	1:N
11	Medición alimenta Corrida de Entrenamiento	N:M
12	Medición utilizada en Predicción	N:M
13	Modelo tiene Corrida de Entrenamiento	1:N
14	Modelo genera Predicción	1:N
15	Corrida de Entrenamiento produce Métrica	1:N
16	Predicción dispara Alerta	1:N

Adicionalmente, **Intervención se compara por similitud con Intervención** (vía embedding sobre el campo de observaciones). No se representa en el diagrama como relación estructural porque es una operación de consulta (búsqueda por similitud).

3.4 Restricciones y decisiones de diseño relevantes

- Un sensor mide una sola variable.
- Una alerta se interpreta con la configuración/umbral vigente al momento de su apertura, no con la configuración actual — por eso Configuración de Dispositivo está historizada.
- Toda intervención pertenece a una orden de trabajo.
- El rol científico de datos no tiene visibilidad de qué técnico específico realizó una intervención.
- Evento depende de Medición: se detecta analizando el valor de una medición concreta, no directamente del sensor como fuente genérica.
- Un evento puede disparar varias alertas y una alerta puede derivar en varias órdenes de trabajo (ambas 1:N).
- Un evento por deficiencia en calidad de medición (medición fuera de rango o ausencia de reporte) se reporta asociada al sensor, nunca al dispositivo, para distinguir una falla de instrumentación de una falla real del equipo.
- Cada alerta tiene un único origen: o un evento detectado por umbral, o una predicción de falla que supera el umbral del modelo — nunca ambos. Un mismo dispositivo puede tener varias alertas abiertas simultáneamente, incluso de orígenes distintos.
- El responsable de una alerta se identifica indirectamente, a través del usuario que realizó la intervención asociada — no hay una relación directa Usuario-Alerta.
- Ubicación se modela con una jerarquía autoreferencial (planta → área → línea), lo que permite resolver a qué planta pertenece cualquier nivel inferior — necesario para el aislamiento de datos por planta (RLS).



Referencias

A continuación se aclaran referencias de grupos. Para detalles sobre los atributos de las entidades ver sección 3. *Modelo conceptual* del informe.



Figura 1: Modelo conceptual

4. Modelo lógico relacional

4.1 Descripción general

El modelo lógico pasa de las 14 entidades presentadas en el modelo conceptual a **22 tablas relacionales**, repartidas en la arquitectura Medallion: bronce (2), plata (13) y oro (7).

Este aumento en la cantidad de elementos se da por:

- Algunos atributos del modelo conceptual como `tipo_dispositivo`, `tipo_variable` y `unidad` ahora son tablas propias.
- Aparecen ahora las tablas de ingesta de datos crudos, no presentes en el conceptual.
- Se agregan dos tablas analíticas a modo de ejemplo (aparte de las asociadas a la predicción): `agregado_horario` y `agregado_diario`.
- Las dos relaciones N:M del conceptual no se convierten en tablas puente. Sino que se resuelven guardando el criterio que selecciona las mediciones (rango temporal y filtros) en lugar de enumerarlas fila por fila.

Volúmenes y retención estimados

Se realiza una breve estimación de volúmenes de datos que surge de contemplar 2 plantas $\times$ 10 dispositivos por planta (3 motores, 3 bombas, 2 cintas transportadoras, 2 tableros eléctricos) = 52 sensores en total, con frecuencias de muestreo que podrían ir de 10 a 60 segundos según el tipo, lo que da unas 161.000 lecturas diarias.

Esto derivaría en aproximadamente 59 millones de registros de mediciones al año. Por lo tanto se plantea: - En la capa plata, sólo se retiene un año ventana de mediciones. - Se retienen todos

los datos históricos de eventos, alertas e intervenciones porque se contempla serán de unas decenas de miles al año. - La capa analítica en oro, se retiene probablemente por algunos años, ya que realiza agrupaciones reduciendo notablemente el volumen y permitiendo hacer análisis interanual y demás. - Las tablas de entrenamiento y predicciones se retienen probablemente por algunos años. - Las tablas tipo catálogos se retienen completas.

Nota sobre el alcance del modelo

Se considera que la base registra hechos; se reserva para capa de aplicaciones cuestiones como el gobierno de los procesos con estado. Ejemplo: el modelo almacena que una alerta se abrió, que una orden de trabajo existe y que una intervención se registró, pero no las transiciones ni las reglas que las producen. Las agrupaciones alertas del mismo dispositivo, asignaciones de técnicos, escalar una alerta o cerrar una orden se contemplan dentro de aplicaciones fuera del alcance de este proyecto.

Adicionalmente, el sistema se plantea **en régimen**, con aproximadamente 18 meses de operación previa: ya existe historia suficiente de mediciones e intervenciones para haber podido entrenar modelos, y entonces el ciclo de predicción estar activo.

4.2 Capa bronce

Recibe el dato tal como llega. Las tablas no tienen claves foráneas. El payload se almacena como jsonb (no json) para poder indexar y consultar claves internas sin parsear el documento completo.

Se separan dos tablas porque los patrones de llegada son considerablemente distintos: las mediciones son de ocurrencia constante, mientras que los cambios de configuración de los sensores son mucho más esporádicos y manuales.

bronze.medicion_cruda

Campo	Tipo	Descripción	Clave
id_medicion_cruda	bigserial	Identificador interno del lote recibido	PK
payload	jsonb	Contenido del mensaje tal como llegó	
timestamp_recepcion	timestampz	Momento de llegada a la base	
procesado	boolean	Indica si la fila ya fue promovida a la capa plata	
error_validacion	text	Motivo del rechazo si no superó la validación; NULL si fue válida	

bronze.configuracion_cruda

Campo	Tipo	Descripción	Clave
id_configuracion_cruda	bigserial	Identificador interno del mensaje recibido	PK
payload	jsonb	Configuración de sensor tal como llegó (umbrales, calibración)	
timestamp_recepcion	timestampz	Momento de llegada a la base	
procesado	boolean	Indica si la fila ya fue promovida a la capa plata	

4.3 Capa plata

Es la capa normalizada del modelo. Se busca asegurar la integridad referencial y las restricciones del dominio. Todas las tablas están en 3FN.

Catálogos

plata.unidad

Campo	Tipo	Descripción	Clave
id_unidad	serial	Identificador de la unidad de medida	PK
simbolo	varchar(10)	Símbolo de la unidad (mm/s, °C, A, bar)	
nombre	varchar(50)	Nombre completo de la unidad	

plata.tipo_variable

Campo	Tipo	Descripción	Clave
id_tipo_variable	serial	Identificador del tipo de variable medida	PK
nombre	varchar(50)	Variable física (vibración, temperatura, corriente, presión, caudal, velocidad, consumo, tensión)	
id_unidad	integer	Unidad en que se expresa esta variable	FK → unidad.id_unidad

plata.tipo_dispositivo

Campo	Tipo	Descripción	Clave
id_tipo_dispositivo	serial	Identificador del tipo de equipo	PK
nombre	varchar(50)	Tipo de equipo (motor eléctrico, bomba centrífuga, cinta transportadora, tablero eléctrico)	
descripcion	text	Descripción del tipo de equipo	

El catálogo podría contener más tipos que los cuatro que se toman de ejemplo para todo el trabajo.

Activos

plata.ubicacion

Campo	Tipo	Descripción	Clave
id_ubicacion	serial	Identificador de la ubicación	PK
nombre	varchar(100)	Nombre de la planta, área o línea	
nivel	varchar(20)	Nivel jerárquico: planta, área o línea	
id_ubicacion_padre	integer	Ubicación contenedora; NULL en el nivel planta	FK → ubicacion.id_ubicacion

La tabla contiene jerarquía autoreferencial. Se puede recorrer hacia arriba permitiendo encontrar finalmente a qué planta pertenece cualquier nivel inferior. Esto será necesario para el aislamiento de información por RLS.

plata.dispositivo

Campo	Tipo	Descripción	Clave
id_dispositivo	serial	Identificador del equipo	PK
nombre	varchar(100)	Identificación del equipo en planta	
id_tipo_dispositivo	integer	Tipo de equipo	FK → tipo_dispositivo.id_tipo_dispos.
id_ubicacion	integer	Ubicación física (típicamente una línea)	FK → ubicacion.id_ubicacion
estado	varchar(20)	Situación del equipo: operativo, en mantenimiento o dado de baja	

Campo	Tipo	Descripción	Clave
fecha_alta	date	Fecha de incorporación al sistema	

El campo estado evita que un equipo detenido de forma planificada genere eventos de falta de reporte.

plata.sensor

Campo	Tipo	Descripción	Clave
id_sensor	serial	Identificador del sensor	PK
id_dispositivo	integer	Equipo al que está montado	FK → dispositivo.id_dispositivo
id_tipo_variable	integer	Variable que mide	FK → tipo_variable.id_tipo_variable
nombre	varchar(100)	Identificación del sensor	
estado	varchar(20)	Activo o fuera de servicio	

plata.configuracion_dispositivo

Campo	Tipo	Descripción	Clave
id_configuracion	serial	Identificador de la versión de configuración	PK
id_dispositivo	integer	Equipo configurado	FK → dispositivo.id_dispositivo
parametros	jsonb	Parámetros de operación configurados y valores	
valido_desde	timestampz	Inicio de vigencia de esta configuración	
valido_hasta	timestampz	Fin de vigencia; NULL en la configuración vigente	

El campo parámetros contiene los umbrales utilizados por el trigger para crear un evento por superación de valor umbral. El trigger se aplica recién en esta etapa y no en bronce porque aquí los datos están validados, evitando disparos de alertas por mediciones corruptas.

No se sobrescriben las versiones anteriores de configuración. Esto porque una alerta pasada debe poder interpretarse con el umbral vigente en su momento, no con el actual, dado que podría prestarse a confusión.

Telemetría

plata.medicion

Campo	Tipo	Descripción	Clave
id_sensor	integer	Sensor que generó la medición	PK, FK → sensor.id_sensor
timestamp_medicion	timestampz	Momento de la medición	PK
valor	numeric(12,4)	Valor medido, en la unidad del tipo de variable	
calidad	varchar(20)	Resultado de la validación: válida, fuera de rango o sospechosa	

Tabla con clave primaria compuesta. Estará particionada por rango mensual sobre timestamp_medicion, con índice BRIN sobre esa columna (ver Actividad 12). Se utiliza formato largo: una fila por lectura, no una columna por variable, para incorporar nuevos sensores sin rediseñar el modelo.

El campo calidad también dispara eventos: una medición marcada como fuera de rango o sospechosa porque viene vacía genera un evento pero apuntado al sensor, no al dispositivo.

plata.evento

Campo	Tipo	Descripción	Clave
id_evento	bigserial	Identificador del evento	PK
id_sensor	integer	Sensor de la medición que lo disparó	FK → sensor.id_sensor
timestamp_medicion	timestampz	Momento de la medición que lo disparó	
tipo_evento	varchar(30)	Umbral superado, valor fuera de rango o ausencia de reporte	
descripcion	text	Descripción legible del evento	
valor_medido	numeric(12,4)	Copia del valor que disparó el evento	
umbral_vigente	numeric(12,4)	Copia del umbral aplicado en ese momento	
timestamp_deteccion	timestampz	Momento en que el trigger registró el evento	

id_sensor y timestamp_medicion identifican la medición de origen, pero **no se declara clave foránea hacia medicion**. Esto dado que las mediciones se borran al año por su alto volumen

y los eventos deben sobrevivir, porque alimentan órdenes de trabajo que se conservan mucho más tiempo. Por el mismo motivo se copian `valor_medido` y `umbral_vigente`, de modo que el evento siga siendo interpretable una vez eliminada la medición que le dio origen.

Los eventos se detectan mediante un trigger sobre la capa plata y no sobre bronce, para que la detección corra sobre datos ya validados. Caso contrario podría dispararse un evento por una medición fuera de rango o inexistente para revisar el dispositivo contralo cuando en realidad lo que hay que revisar es el sensor.

plata.alerta

Campo	Tipo	Descripción	Clave
<code>id_alerta</code>	<code>bigserial</code>	Identificador de la alerta	PK
<code>id_dispositivo</code>	<code>integer</code>	Equipo afectado	FK → <code>dispositivo.id_dispositivo</code>
<code>id_sensor</code>	<code>integer</code>	Sensor afectado, solo en alertas de instrumentación; NULL en el resto	FK → <code>sensor.id_sensor</code>
<code>id_evento</code>	<code>bigint</code>	Evento que la originó; NULL en alertas predictivas	FK → <code>evento.id_evento</code>
<code>id_prediccion</code>	<code>bigint</code>	Predicción que la originó; NULL en alertas por evento	FK → <code>oro.prediccion.id_prediccion</code>
<code>origen</code>	<code>varchar(20)</code>	Umbral o predictivo	
<code>severidad</code>	<code>varchar(20)</code>	Nivel de criticidad	
<code>estado</code>	<code>varchar(20)</code>	Abierta, en revisión, cerrada	
<code>timestamp_apertura</code>	<code>timestampz</code>	Momento de creación de la alerta	
<code>timestamp_cierre</code>	<code>timestampz</code>	Momento de cierre; NULL mientras está abierta	

Restricción CHECK: exactamente uno entre `id_evento` e `id_prediccion` debe estar presente. Cada alerta tiene un único origen, aunque un mismo dispositivo puede tener varias alertas abiertas simultáneamente, incluso de orígenes distintos.

`id_prediccion` es la única clave foránea que apunta de plata hacia oro; todas las demás dependencias entre capas van en sentido contrario.

Gestión

plata.usuario

Campo	Tipo	Descripción	Clave
<code>id_usuario</code>	<code>serial</code>	Identificador del usuario de aplicación	PK
<code>nombre</code>	<code>varchar(100)</code>	Nombre del usuario	
<code>email</code>	<code>varchar(100)</code>	Correo de contacto	

Campo	Tipo	Descripción	Clave
rol_negocio	varchar(30)	Rol (operario, técnico, supervisor, científico de datos, administrador)	
id_ubicacion	integer	Ámbito de alcance del usuario	FK → ubicacion.id_ubicacion

Los usuarios de aplicación no son roles de PostgreSQL: la aplicación se conecta con un rol de servicio y comunica la identidad por variable de sesión, que las políticas RLS luego consultan (ver Actividad 11).

plata.orden_trabajo

Campo	Tipo	Descripción	Clave
id_orden_trabajo	serial	Identificador de la orden	PK
id_alerta	bigint	Alerta que la originó	FK → alerta.id_alerta
tipo	varchar(30)	Correctiva, preventiva o de instrumentación	
estado	varchar(20)	Situación de la orden	
fecha_apertura	date	Fecha de emisión	
fecha_cierre	date	Fecha de cierre; NULL mientras está en curso	

La orden no almacena el equipo ni el sensor objetivo: ambos se obtienen de la alerta que la originó, evitando una dependencia transitiva.

plata.intervencion

Campo	Tipo	Descripción	Clave
id_intervencion	serial	Identificador de la intervención	PK
id_orden_trabajo	integer	Orden a la que pertenece	FK → orden_trabajo.id_orden_trabajo
id_usuario	integer	Técnico que la realizó	FK → usuario.id_usuario
fecha	date	Fecha de ejecución	
observaciones	text	Descripción en lenguaje natural de lo realizado y observado	
embedding	vector(384)	Representación vectorial de las observaciones, para búsqueda por similitud	

Toda intervención pertenece a una orden de trabajo. El campo embedding (extensión pgvector) permite buscar intervenciones pasadas con descripciones parecidas; se desarrolla en la Actividad 9. El rol científico de datos no accede a id_usuario, por tratarse de un dato personal innecesario para el análisis.

4.4 Capa oro

Esta capa contiene una capa de consumo analítico, desnormalizada y optimizada para lectura. No recibe escrituras directas de la operación: se recalcula a partir de plata.

Luego, convive en esta capa un bloque asociado a la predicción (modelo, corrida_entrenamiento, metrica, prediccion) que sí está normalizado, por ser metadatos de bajo volumen y no datos analíticos agregados y donde la redundancia introduciría anomalías de actualización.

Agregados analíticos

oro.agregado_horario

Campo	Tipo	Descripción	Clave
id_sensor	integer	Sensor agregado	PK, FK → plata.sensor.id_sensor PK
hora	timestampz	Hora truncada del período agregado	
promedio	numeric(12,4)	Valor promedio en la hora	
minimo	numeric(12,4)	Valor mínimo en la hora	
maximo	numeric(12,4)	Valor máximo en la hora	
desvio	numeric(12,4)	Desvío estándar en la hora	
cantidad_lecturas	integer	Lecturas efectivamente registradas	

oro.agregado_diario

Campo	Tipo	Descripción	Clave
id_sensor	integer	Sensor agregado	PK, FK → plata.sensor.id_sensor PK
dia	date	Día del período agregado	
promedio	numeric(12,4)	Valor promedio en el día	
minimo	numeric(12,4)	Valor mínimo en el día	
maximo	numeric(12,4)	Valor máximo en el día	
desvio	numeric(12,4)	Desvío estándar en el día	

Campo	Tipo	Descripción	Clave
cantidad_lecturas	integer	Lecturas efectivamente registradas	

Ambos agregados sostienen los tableros de supervisión y el análisis histórico. Se conservan más allá del año de retención del detalle en plata, lo que permite análisis interanual sin sostener la serie completa de mediciones en plata.

Features para el modelo predictivo

Las columnas que se presentan a continuación tienen fines ilustrativos. El objetivo es mostrar el formato ancho que requiere el consumo por parte de un modelo de aprendizaje automático y su relación con el resto del esquema, no proponer un conjunto de features validado que realmente fuera a funcionar, lo cual excede el alcance de este trabajo. Lo mismo aplica al esquema de entrenamiento y predicción planteado: interesa cómo se estructuran y se relacionan los datos, no la eficacia del modelo.

oro.feature_motor_ventana

Campo	Tipo	Descripción	Clave
id_dispositivo	integer	Dispositivo en cuestión	PK, FK → plata.dispositivo.id_dispositivo
ventana_hasta	timestampz	Fin de la ventana temporal resumida	PK
ventana_desde	timestampz	Inicio de la ventana temporal resumida	
vib_media	numeric(12,4)	Vibración promedio en la ventana	
vib_max	numeric(12,4)	Vibración máxima en la ventana	
vib_desvio	numeric(12,4)	Desvío de la vibración en la ventana	
vib_tendencia	numeric(12,4)	Pendiente de la vibración: captura si viene creciendo	
temp_media	numeric(12,4)	Temperatura promedio en la ventana	
temp_max	numeric(12,4)	Temperatura máxima en la ventana	
corr_media	numeric(12,4)	Corriente promedio en la ventana	
cant_eventos	integer	Eventos registrados en la ventana	
horas_operacion	numeric(6,2)	Horas efectivas de operación en la ventana	

Campo	Tipo	Descripción	Clave
fallo_en_horizonte	boolean	Etiqueta de entrenamiento: si hubo intervención correctiva dentro del horizonte posterior. NULL mientras el horizonte no transcurrió	
timestamp_calculo	timestampz	Momento en que se calculó la fila	

Una fila por dispositivo y ventana. La ventana es de **24 horas y se recalcula cada hora**, de modo que la tabla crece unas 175 mil filas al año, frente a los ~59 millones de medicion.

La ventana y el horizonte son dos parámetros distintos. La ventana define cuánto se mira hacia atrás para resumir el comportamiento del equipo (24 horas). El horizonte define cuánto se mira hacia adelante al predecir: se adopta un valor inicial de 72 horas, tiempo razonable para planificar una intervención antes de la falla.

De ahí surge el comportamiento de fallo_en_horizonte. Cuando la fila se calcula, todavía no se sabe qué va a pasar en las 72 horas siguientes, con lo cual la etiqueta queda en NULL y esa fila es sobre la que se predice. Transcurrido el horizonte, se verifica si hubo una intervención correctiva sobre el equipo en ese lapso y se carga la etiqueta, momento en que la fila pasa a estar disponible para entrenar. La tabla queda así dividida en filas maduras, que sirven para entrenamiento, y filas recientes, sobre las que se predice.

Esta única tabla alimenta tanto el entrenamiento como la predicción, lo que garantiza que las features se calculen de forma idéntica en ambos casos. La diferencia está en qué filas usa cada uno: las anteriores al horizonte de predicción ya tienen fallo_en_horizonte cargado y sirven para entrenar; las más recientes lo tienen en NULL y son sobre las que se predice, como se mencionó anteriormente.

La clave es compuesta con id_dispositivo y ventana_hasta. Esta combinación será utilizada luego desde la tabla oro.prediccion para poder identificar la prediccion a la fila de feature_motor_ventana utilizada. Ver notas en oro.prediccion.

fallo_en_horizonte se almacena como columna en lugar de calcularse en cada entrenamiento. oro.feature_bomba_ventana, oro.feature_cinta_ventana y oro.feature_tablero_ventana son estructuras análogas, con las columnas correspondientes a sus propios sensores:

Tabla	Columnas específicas
feature_bomba_ventana	presión (media, máx, desvío, tendencia), caudal (media, desvío), temperatura (media, máx)
feature_cinta_ventana	velocidad (media, desvío, tendencia), corriente (media, máx)
feature_tablero_ventana	consumo (media, máx, tendencia), tensión (media, desvío)

Se mantienen tablas separadas por tipo de dispositivo porque las variables medidas no son comparables entre tipos: una tabla única dejaría la mayoría de las columnas en NULL para cada fila, y un valor nulo en una feature no es neutro para un modelo de aprendizaje automático.

Trazabilidad del modelo predictivo

oro.modelo

Campo	Tipo	Descripción	Clave
id_modelo	serial	Identificador del modelo	PK
nombre	varchar(100)	Nombre del modelo	
version	varchar(20)	Versión	
id_tipo_dispositivo	integer	Tipo de equipo al que aplica	FK → plata.tipo_dispositivo.id_tipo_c
horizonte_h	integer	Horizonte de predicción en horas	
estado	varchar(20)	Activo o retirado	

Existe un modelo por tipo de dispositivo, ya que la evolución de un motor hacia su deterioro no es comparable por ejemplo con la de un tablero eléctrico. Los modelos retirados se conservan para poder interpretar predicciones históricas.

oro.corrida_entrenamiento

Campo	Tipo	Descripción	Clave
id_corrida	bigserial	Identificador de la corrida	PK
id_modelo	integer	Modelo entrenado	FK → modelo.id_modelo
fecha	timestampz	Momento del entrenamiento	
rango_desde	timestampz	Inicio del período de datos utilizado	
rango_hasta	timestampz	Fin del período de datos utilizado	
criterio_seleccion	jsonb	Filtros aplicados para seleccionar las filas de features	
hiperparametros	jsonb	Configuración del algoritmo, en formato flexible	
uri_artefacto	text	Ruta al modelo entrenado, almacenado fuera de la base	

rango_desde, rango_hasta y criterio_seleccion reemplazan la tabla puente que exigiría la relación N:M del modelo conceptual: la corrida queda reproducible sin enumerar las mediciones que participaron.

El uso de jsonb para los hiperparámetros mantiene el modelo independiente del algoritmo concreto que se aplique.

oro.metrica

Campo	Tipo	Descripción	Clave
id_metrica	bigserial	Identificador de la métrica	PK
id_corrida	bigint	Corrida que la produjo	FK → corrida_entrenamiento.id_corrida
nombre	varchar(50)	Nombre de la métrica	
particion	varchar(20)	Conjunto sobre el que se calculó: entrenamiento, validación o test	
valor	numeric(12,6)	Valor obtenido	

Formato largo: una fila por métrica en lugar de una columna por métrica. Permite registrar cualquier indicador sin modificar el esquema, sea cual sea el tipo de modelo aplicado.

oro.prediccion

Campo	Tipo	Descripción	Clave
id_prediccion	bigserial	Identificador de la predicción	PK
id_dispositivo	integer	Equipo evaluado	FK → plata.dispositivo.id_dispositivo
id_modelo	integer	Modelo que la generó	FK → modelo.id_modelo
timestamp_prediccion	timestampz	Momento en que se ejecutó la predicción	
ventana_desde	timestampz	Inicio de la ventana de features utilizada	
ventana_hasta	timestampz	Fin de la ventana de features utilizada	
horizonte_h	integer	Horas hacia adelante que cubre la predicción	
score	numeric(5,4)	Probabilidad estimada de falla	
umbral_aplicado	numeric(5,4)	Umbral vigente al momento de la predicción	

La predicción se ejecuta cada hora por dispositivo, sobre las filas de features más recientes. Un trigger genera una alerta de origen predictivo cuando score supera umbral_aplicado, generando una alerta → orden de trabajo. El trigger sólo crea la alerta: su gestión posterior corresponde a la aplicación.

Guardar umbral_aplicado junto al score permite que una predicción antigua siga siendo interpretable aunque el criterio de alerta cambie después.

id_dispositivo y ventana_hasta identifican la fila de features utilizada, pero **no se declara clave foránea**, porque la tabla de destino depende del tipo de dispositivo. El vínculo se resuelve por JOIN contra la tabla de features correspondiente.

Un score numérico acompañado de su umbral es la salida natural de prácticamente cualquier clasificador, de modo que el esquema no queda atado a una técnica concreta de aprendizaje automático.

4.5 Diagrama

El diagrama entidad-relación del modelo lógico, con todas las tablas, claves y relaciones, se adjunta como `docs/modelo_logico.png`. Además se proporciona el acceso directo `docs/modelo_logico_interactivo` a la URL de `dbdiagram` que permite visualizar los elementos del modelo de manera interactiva. Finalmente, se agrega el código que genera el modelo en `dbdiagram` en `anexos/modelo_logico_interactivo.md`.



Figura 2: Modelo lógico

5. Normalización, desnormalización y decisiones de diseño

5.1 Criterio general

El nivel de normalización no es el mismo para todo el modelo. Como criterio general adoptado: **se normaliza donde se escribe, y se desnormaliza lo que es derivado y voluminoso.**

Se normaliza para eliminar redundancias (mismo dato repetido en varias filas de distintas tablas) cuando son tablas en las que se escribe recurrentemente por diseño porque es en esos casos es cuando la redundancia presenta peligro. Si dicho valor repetido comienza a actualizarse en todas las filas y tablas, y hay un fallo que deja algunas filas actualizadas y otras no, la base queda en estado contradictorio y puede ser complejo reconocer el valor correcto y recuperarlo correctamente.

Por eso la capa plata, que recibe constantemente inserciones y modificaciones desde la operación, está de manera general normalizada en 3FN.

Por otro lado, las tablas agregadas (analíticas) y las features (para predicción) de la capa oro se desnormalizan: son datos calculados a partir de plata, nadie los edita a mano, y si se pierden se recalculan (a diferencia de la complejidad de restaurar un valor parcialmente actualizado en la capa de plata).

La capa oro, sin embargo, contiene dos bloques desnormalizados y uno normalizado:

Bloque	Contenido	Estructura
Agregados	agregado_horario, agregado_diario	Desnormalizada
Features	feature_motor_ventana y análogas	Desnormalizada
Trazabilidad	modelo, corrida_entrenamiento, metrica, prediccion	Normalizada

Los dos primeros son datos derivados de medición y de gran volumen, con lo cual se aplica el criterio de desnormalización (ya que el gran volumen invita a evitar JOINS a causa de la normalización que terminan siendo “caros”). El bloque de trazabilidad no implica campos calculados ni gran volumen: no se deriva de ninguna otra tabla, se escribe directamente (cada entrenamiento inserta una corrida, cada hora se inserta una predicción). Ninguno de los dos motivos para desnormalizar aplica, y por eso se mantiene normalizado.

La capa bronce, por otro lado, no le aplica nada de esto, ya que como está planteada, en bronce el dato se almacena como jsonb sin esquema declarado, con lo cual no hay dependencias para evaluar. La capa simplemente existe para poder recibir el dato tal como llega, incluso cuando pudiera no cumplir alguna reglas del dominio.

5.2 Normalización en la capa plata

Todas las tablas de plata están en 3FN. El criterio se aplicó de manera general, verificando en cada tabla que todos sus atributos dependan de la clave primaria y no de otro atributo no clave.

Un caso que ejemplifica claramente esto es el de la unidad de medida. En el modelo conceptual, la entidad sensor incluía los atributos tipo_variable y unidad. Pero si eso se llevara a tabla así como está, se incumpliría la tercera forma normal, porque en realidad la unidad no depende del sensor sino del tipo de variable: toda medición de temperatura se expresa en la misma unidad, sea cual sea el sensor que la tome.

En el modelo lógico la unidad se separa y pasa al catálogo `tipo_variable`. Así se evitan problemas de redundancia con los riesgos de anomalías en actualizaciones, inserciones, etc.

5.3 Normalización y desnormalizado en la capa oro

En los agregados y las features aplicamos desnormalización ya que se almacena un cálculo en lugar de calcularlo a demanda en la consulta. `agregado_diario.promedio` no es un hecho sino una agregación tipo `AVG()` sobre `medicion`.

Esto muestra que no se cumple la dependencia funcional mencionada en el punto anterior que exige la 3FN. El valor de promedio no depende estrictamente de la clave (`id_sensor`, `dia`), sino de las aproximadamente 2.880 mediciones de ese día. La clave etiqueta el valor pero no lo determina, depende de otra cosa.

De alguna manera el mismo dato existe dos veces y en dos formas distintas (una especie de redundancia). Las mediciones de un día están completas en `medicion` y resumidas en una fila de `agregado_diario`. Pero no presenta el riesgo de redundancia de la capa plata. Si por algo el cálculo de la fila agregada fallara, se vuelve a calcular a partir de plata, que tiene integridad asegurada.

Las features agregan la desnormalización típica necesaria para ML: el formato ancho. Mientras `medicion` guarda una fila por lectura sin importar qué variable sea, `feature_motor_ventana` tiene `vib_media`, `temp_media` y `corr_media` como columnas separadas. La variable dejó de ser un dato para pasar a ser parte de la estructura, y por eso incorporar un sensor nuevo a un motor obliga a agregar columnas, cosa que en `medicion` no pasa.

En ambos casos se asume que los datos quedan desactualizados hasta el siguiente recálculo. Pero como ventaja, las consultas analíticas no tienen que recorrer millones de filas.

5.4 Excepciones de Desnormalizaciones en plata

Hay tres casos que se apartan de la 3FN en esta capa a propósito.

`evento` copia `valor_medido` y `umbral_vigente`, que en realidad ya están en `medicion` y en `configuracion_dispositivo`. Esto porque las mediciones se borran al año y los eventos se conservan mucho más tiempo. Si no se hiciera la copia quedarían eventos en los que se sabría que hubo una superación de umbral pero no de cuánto ni contra qué límite porque ya no tendrías la medición para ir a buscar esos datos. El compromiso es que se renuncia a la clave foránea hacia `medicion` (para poder borrar los registros de mediciones con libertad), aunque no hay riesgo de inconsistencia porque ambos valores no tienen alteración una vez ocurrido el hecho.

`alerta` guarda `id_dispositivo` aunque sea derivable con algunos saltos entre tablas. El motivo es que el camino hasta el dispositivo depende del origen de la alerta: si nació de un evento, hay que recorrer `alerta` → `evento` → `sensor` → `dispositivo`; si nació de una predicción, `alerta` → `prediccion` → `dispositivo`. Sin la columna agregada en redundancia, determinar a qué equipo pertenece una alerta requeriría dos recorridos alternativos según cuál clave foránea esté cargada (evento o predicción). Como las políticas RLS se evalúan fila por fila en toda consulta sobre la tabla, ese recorrido condicional se repetiría en cada evaluación. Con la columna, la política resuelve la planta con una sola comparación. Hay una redundancia pero es acotada porque el valor se conoce al crear la alerta y no cambia nunca.

`intervencion` almacena `embedding`, que es un dato derivado del texto de observaciones. Por el criterio general correspondería calcularlo en el momento de la búsqueda o ubicarlo en la capa oro por ser derivado, pero se guarda junto al texto porque calcular un `embedding` requiere

invocar un modelo externo y sería inviable hacerlo directamente en cada consulta. Dado que las observaciones no se modifican una vez registrada la intervención, el embedding no puede quedar desactualizado, por lo que el riesgo es mínimo.

5.5 Resolución de las relaciones N:M

El modelo conceptual define dos relaciones muchos a muchos: Medición ↔ Corrida de Entrenamiento y Medición ↔ Predicción. Lo típico sería una tabla puente que enumere cada par.

Pero con unos 59 millones de mediciones al año, una corrida entrenada sobre un año de datos generaría 59 millones de filas puente, y doce corridas anuales superarían los 700 millones de filas cuya única función sería registrar qué mediciones participaron.

Entonces, en lugar de eso la relación se guarda por comprensión: `corrida_entrenamiento` almacena `rango_desde`, `rango_hasta` y `criterio_seleccion`, de modo que el conjunto de mediciones queda determinado por esa regla en vez de enumerado. `prediccion` resuelve lo mismo con `ventana_desde` y `ventana_hasta`.

El compromiso es que se pierde integridad referencial hacia las mediciones concretas, que de todas formas se borran al año.

5.6 Correspondencia con los patrones de consulta

La estructura elegida responde a cuatro formas de consumo bien distintas.

La aplicación, que es por donde operan operarios y técnicos, escribe alertas, órdenes e intervenciones y lee el estado actual, siempre bajo RLS. Consulta pocas filas por vez y necesita que el dato sea exacto, con lo cual la normalización de plata le garantiza consistencia. Además es el consumidor que motiva una de las dos desnormalizaciones de esa capa: `id_dispositivo` en alerta existe para que RLS resuelva la planta sin recorridos condicionales.

El supervisor puede consultar la base directamente. Usa plata para el estado operativo (alertas abiertas, últimas lecturas) y oro para tendencias, con lo cual atraviesa las dos capas y aprovecha las dos estructuras.

El científico de datos trabaja sobre oro, con análisis histórico de meses o años sobre todas las plantas y sin acceso a datos personales. Los precálculos de las tablas de fetures le evita agregar la serie completa en cada consulta y le brinda el formato “ancho” que necesita para sus modelos, consumidos a través del servicio de predicción.

6. Selección tecnológica

6.1 Descripción general

En esta sección se justifica la elección del motor de base de datos sobre el que se implementa el sistema. La decisión no se toma en abstracto, sino a partir de los requisitos técnicos que impone el caso de monitoreo IoT con análisis predictivo, contrastando la opción adoptada contra las alternativas razonables que se evaluaron.

La conclusión, que se desarrolla a lo largo de la sección, es el uso de un **motor único: PostgreSQL, con la extensión pgvector**. La particularidad del caso es que combina tipos de datos y patrones de acceso que suelen resolverse en sistemas distintos y especializados, y la pregunta de fondo es si conviene combinar varios motores (uno por necesidad) o cubrir todo con uno solo lo suficientemente versátil.

6.2 Requisitos técnicos que impone el caso

Antes de comparar tecnologías conviene enumerar qué necesidades concretas tiene que satisfacer la base, porque son esas necesidades las que definen contra qué se compara cada candidato. Del dominio surgen cinco requisitos que no siempre conviven en un mismo motor:

1. Series temporales de alto volumen. El corazón del sistema son las mediciones de los sensores: del orden de 59 millones de filas al año (ver Actividad 12), llegando de forma continua y consultándose siempre por rango temporal (últimas lecturas, promedios por día, medias móviles). Esto exige escritura sostenida, particionamiento e índices adecuados para datos ordenados por tiempo.

2. Datos semiestructurados (JSON). Tanto el payload crudo que llega a la capa bronce como la configuración de los sensores (umbrales, calibración) tienen forma de documento con estructura variable. Se necesita poder almacenarlos sin un esquema rígido y, a la vez, poder consultar e indexar claves internas sin parsear el documento completo cada vez.

3. Texto no estructurado con búsqueda por similitud. Las observaciones que deja el técnico en cada intervención son texto libre en lenguaje natural. El caso pide poder recuperar intervenciones pasadas con síntomas parecidos, lo que implica trabajar con *embeddings* (representaciones vectoriales del texto) y hacer búsqueda por cercanía entre vectores.

4. Aislamiento de datos por planta (seguridad a nivel de fila). Al haber dos plantas y roles con alcance acotado, la base tiene que garantizar que ciertos usuarios solo vean las filas que les corresponden. No es un filtro que pueda quedar solo en la aplicación: se busca resolverlo en el propio motor, con seguridad a nivel de fila (Row-Level Security, RLS; ver Actividad 11).

5. Integridad referencial fuerte en el dominio de gestión. Toda la cadena de negocio —dispositivo, sensor, evento, alerta, orden de trabajo, intervención, usuario— tiene relaciones que deben mantenerse consistentes. Una orden de trabajo no puede apuntar a una alerta inexistente, ni una intervención a una orden que no existe. Esto es terreno natural del modelo relacional con claves foráneas y restricciones (CHECK, UNIQUE).

A esto se suma un requisito transversal: la arquitectura Medallion (bronce / plata / oro) elegida en la Actividad 4 se implementa como *schemas* dentro de un mismo sistema, lo que ya inclina la decisión hacia un motor que permita separar capas lógicamente sin multiplicar la infraestructura.

6.3 La disyuntiva de fondo: motor único vs. persistencia poliglota

Los cinco requisitos anteriores rara vez brillan todos en el mismo producto. Frente a esto hay dos estrategias posibles.

La primera es la **persistencia poliglota**: usar varios motores, cada uno especializado en lo que mejor hace. Por ejemplo, una base de series temporales (InfluxDB, TimescaleDB) para las mediciones, una base documental (MongoDB) para el JSON, una base vectorial dedicada (Pinecone, Milvus, Qdrant) para los embeddings, y una relacional para la gestión. Cada pieza rinde muy bien en su terreno.

La segunda es el **motor único**: elegir un sistema lo bastante versátil como para cubrir todos los requisitos de manera aceptable, aunque en algún punto individual no sea el mejor del mercado.

El costo de la opción poliglota no aparece en el rendimiento de cada motor, sino en el sistema como conjunto:

Dimensión	Costo de combinar varios motores
Integridad referencial	Se pierde entre sistemas. Una alerta en un motor no puede tener una clave foránea a una predicción de otro motor; la consistencia queda librada a la aplicación.
Seguridad y aislamiento	Hay que replicar el control de acceso (RLS) en cada motor, con criterios y mecanismos distintos, multiplicando la superficie de error.
Operación	Más piezas que desplegar, respaldar, monitorear, versionar y mantener sincronizadas. Cada motor es un punto de falla y una dependencia más.
Consultas mixtas	Cruzar una intervención (texto/vector) con su orden de trabajo y su dispositivo (relacional) obliga a unir datos entre sistemas en la aplicación, en vez de un simple JOIN.

Para el volumen y la complejidad de este caso, esos costos superan a la ganancia de rendimiento por especialización. El sistema no maneja el volumen de texto ni la escala de series que justificarían un motor dedicado; sí necesita, en cambio, integridad y seguridad coherentes en todo el dominio, que es precisamente lo que la opción poliglota vuelve difícil. Por eso se opta por un motor único, siempre que exista uno capaz de cubrir los cinco requisitos.

6.4 La opción elegida: PostgreSQL + pgvector

PostgreSQL es una base de datos relacional de código abierto, madura y extensible. Esa extensibilidad es la clave de la elección: además de todo lo que se espera de una relacional (claves foráneas, restricciones, transacciones ACID), incorpora de forma nativa o vía extensiones las capacidades que el caso necesita. La siguiente tabla mapea cada requisito de la sección 6.2 con el mecanismo de PostgreSQL que lo resuelve:

Requisito del caso	Cómo lo cubre PostgreSQL
Series temporales de alto volumen	Particionamiento declarativo por rango sobre el timestamp e índices BRIN , muy compactos para datos naturalmente ordenados por tiempo (ver Actividades 7 y 12).
Datos semiestructurados (JSON)	Tipo de dato JSONB , que almacena el documento en forma binaria indexable y permite consultar claves internas sin recorrer todo el texto.
Texto no estructurado + búsqueda vectorial	Extensión pgvector , que agrega el tipo vector y operadores de distancia, con índices aproximados (HNSW / IVFFlat) para búsqueda por similitud (ver Actividad 9).
Aislamiento por planta	Row-Level Security nativo: políticas que filtran filas según la identidad del usuario, evaluadas por el propio motor en cada consulta (ver Actividad 11).
Integridad referencial del dominio	Modelo relacional clásico: claves foráneas, CHECK, UNIQUE, transacciones. Es el terreno propio de PostgreSQL.
Separación en capas (Medallion)	Schemas (bronce, plata, oro) dentro de la misma base, sin necesidad de sistemas separados.

El punto decisivo es que la extensión **pgvector** es lo que permite que la búsqueda por similitud, que sería el argumento más fuerte a favor de sumar un motor dedicado, se resuelva dentro del mismo PostgreSQL. Con eso, el último requisito que empujaba hacia la persistencia poliglota queda cubierto sin salir del motor único, y se conservan la integridad referencial y la seguridad unificadas.

Un beneficio adicional, propio de mantener todo en un solo sistema, es que la búsqueda vectorial puede quedar sujeta a las mismas políticas RLS que el resto de los datos: una búsqueda por similitud sobre las intervenciones no puede devolver resultados de una planta a la que el usuario no tiene acceso (ver Actividades 9 y 11). En un esquema poliglota, con los vectores en un motor aparte, garantizar eso sería considerablemente más complejo.

6.5 Alternativas evaluadas y descartadas

Se consideraron las siguientes alternativas antes de adoptar la opción final.

Bases de series temporales especializadas (TimescaleDB, InfluxDB). Son excelentes para la parte de mediciones: compresión, agregación por ventanas y retención automática están pensadas para exactamente ese tipo de dato. El problema es que resuelven bien *solo* una de las cinco necesidades. El resto del dominio —la integridad referencial de la gestión, el aislamiento por fila, la búsqueda vectorial— queda fuera de su fortaleza. En el caso particular de TimescaleDB, al ser una extensión de PostgreSQL, podría incorporarse más adelante si el volumen de la serie lo exigiera, sin cambiar de motor; se deja anotado como evolución posible, no como necesidad del alcance actual.

Base documental (MongoDB). Encaja muy bien con los datos semiestructurados (el JSON crudo de bronce), pero al costo de perder lo que este dominio más necesita: la integridad referencial entre entidades y el control de acceso a nivel de fila. Modelar la cadena evento → alerta → orden de trabajo → intervención sin claves foráneas dejaría la consistencia

enteramente en manos de la aplicación, que es justamente lo que se quiere evitar. Además, PostgreSQL ya cubre lo semiestructurado con JSONB, con lo cual la ventaja de MongoDB no compensa lo que se resigna.

Base vectorial dedicada (Pinecone, Milvus, Qdrant). Son la mejor opción cuando el volumen de vectores es enorme (millones de documentos, búsqueda semántica a gran escala). En este caso el texto vectorizado se limita a las observaciones de las intervenciones, que son del orden de decenas de miles de registros. Ese volumen lo maneja pgvector sin problemas, de modo que sumar un sistema entero solo para los vectores sería sobredimensionar la solución e introducir todos los costos de la persistencia poliglota (integridad, seguridad, operación) sin una ganancia real.

En síntesis, cada alternativa resolvía bien una arista del problema pero debilitaba las demás. PostgreSQL con pgvector es la única opción evaluada que cubre las cinco necesidades de manera aceptable dentro de un solo sistema, conservando integridad y seguridad coherentes en todo el dominio.

6.6 Relación con el contenido de la materia

La decisión se apoya directamente en temas vistos en clase:

- **Clase 4 (mapa de decisión NoSQL).** El criterio para descartar MongoDB y las bases dedicadas surge de ese mapa: NoSQL conviene cuando se resigna integridad referencial a cambio de escala o flexibilidad de esquema, y este caso valora la integridad por encima de esa escala.
- **Clase 5 (arquitectura Medallion, Data Lake / Warehouse / Lakehouse).** Implementar bronce / plata / oro como schemas de un mismo PostgreSQL corresponde a un *lakehouse lógico* dentro de un único motor, en lugar de sistemas físicamente separados.
- **Clase 6 (bases vectoriales y escalabilidad).** Respalda el uso de pgvector y el criterio de por qué, al volumen de texto de este caso, no se justifica una base vectorial dedicada.

6.7 Conclusión

El caso combina cinco requisitos técnicos —series temporales de alto volumen, datos semiestructurados, búsqueda vectorial sobre texto, aislamiento por fila e integridad referencial— que en un enfoque poliglota exigirían varios motores especializados, a costa de perder consistencia y seguridad unificadas y de multiplicar la complejidad operativa. **PostgreSQL con la extensión pgvector** cubre los cinco dentro de un único sistema: aporta el modelo relacional para la integridad, JSONB para lo semiestructurado, particionamiento y BRIN para la serie, RLS para el aislamiento y pgvector para la búsqueda por similitud. Por eso se adopta como motor único del proyecto, dejando anotado que la incorporación de TimescaleDB —al ser una extensión del mismo PostgreSQL— queda disponible como evolución si el volumen de mediciones llegara a exigirlo, sin necesidad de cambiar de tecnología.

7. Modelo físico e implementación

7.1 Descripción general

Esta sección documenta la implementación física del modelo lógico sobre PostgreSQL. Se pasa de las 22 tablas descritas en la Actividad 4 al DDL concreto (tipos de dato, claves, restricciones), el particionado de la serie temporal, los índices, los triggers que generan eventos y alertas, y una carga de datos de ejemplo coherente que permite validar todo el circuito.

La implementación se probó de punta a punta reconstruyendo la base desde cero sobre **PostgreSQL 16 con la extensión pgvector 0.6.0** (la elegida en la Actividad 6). Todos los scripts corren en orden sin errores y la carga de ejemplo deja la base en un estado consistente sobre el que se ejecutan las consultas de la Actividad 8.

7.2 Organización de los scripts

El código se versiona en la carpeta db/ del repositorio, dividido en subcarpetas por función, con un prefijo numérico que fija el orden de ejecución de punta a punta:

Carpeta	Scripts	Contenido
db/estructura/	00 – 08, 11	Schemas, extensión, DDL de las 22 tablas y triggers
db/indices_vistas/	09, 10	Particiones de medicion e índices
db/datos/	12 – 18	Carga de catálogos, activos, mediciones, agregados/features, trazabilidad, gestión y bronce

Numerarlos permite levantar la base entera desde cero con un solo comando y volver a hacerlo cuantas veces haga falta.

7.3 Orden de creación y dependencias

El orden de creación no es libre: lo imponen las claves foráneas. Una tabla no puede referenciar a otra que todavía no existe. El orden adoptado es:

1. Schemas y extensión (00).
2. Catálogos de plata (01): no dependen de nada.
3. Activos de plata (02): dependen de los catálogos.
4. Telemetría de plata (03): medicion y evento, dependen de sensor.
5. Predicción de oro (04): modelo y prediccion.
6. Alerta de plata (05).
7. Gestión de plata (06): usuario, orden_trabajo, intervencion.
8. Resto de oro (07) y bronce (08).

El punto no evidente está entre los pasos 4, 5 y 6. plata.alerta tiene una clave foránea hacia oro.prediccion —la única dependencia de plata hacia oro en todo el modelo—, así que oro.prediccion debe crearse **antes** que plata.alerta. No es un ciclo, pero obliga a intercalar la creación de esas dos tablas de oro en medio de la capa plata.

7.4 Particionado de medicion

medicion se declara PARTITION BY RANGE (timestamp_medicion): en lugar de una sola tabla, PostgreSQL la divide físicamente en subtablas por rango de fecha. Se crea una partición por

mes (medicion_2026_01 ... medicion_2026_12) con un bloque D0 que las genera en un bucle, en vez de escribir doce CREATE TABLE a mano (09_particiones.sql).

Dos consecuencias prácticas:

- PostgreSQL exige que la columna de particionado forme parte de la clave primaria; por eso la PK es compuesta (id_sensor, timestamp_medicion).
- Cada INSERT se enruta automáticamente a la partición cuyo rango contiene su timestamp. Si esa partición no existe, el INSERT falla. Por eso las particiones se crean **antes** de cargar datos.

El particionado es lo que hace viable el volumen estimado (del orden de 59 millones de filas al año, ver Actividad 12): una consulta acotada a un rango de fechas sólo lee las particiones involucradas, y la purga del año vencido se resuelve descartando particiones enteras en vez de un DELETE masivo. La justificación de rendimiento se desarrolla en la Actividad 12.

7.5 Restricciones e índices

Además de las claves, el esquema declara restricciones que hacen cumplir reglas del dominio a nivel de base:

- CHECK de dominio cerrado en varios campos de texto (ubicacion.nivel, dispositivo.estado, alerta.estado, metrica.particion, etc.): la base rechaza valores fuera del conjunto admitido.
- CHECK de rango en prediccion.score y umbral_aplicado (deben estar entre 0 y 1).
- Doble CHECK en alerta que garantiza el origen único: exactamente una de id_evento / id_prediccion presente, y coherencia con el campo origen.
- Índice único parcial en configuracion_dispositivo: a lo sumo una configuración vigente (valido_hasta IS NULL) por dispositivo, permitiendo muchas versiones históricas.

Los índices (10_indices.sql) responden a los patrones de consulta:

- **BRIN** sobre medicion(timestamp_medicion): mucho más compacto que un B-tree para datos naturalmente ordenados por tiempo. Se propaga a todas las particiones.
- **B-tree** sobre alerta(id_dispositivo, estado) para la consulta de alertas abiertas por planta, y sobre evento(id_sensor, timestamp_medicion).
- Índice **parcial** sobre feature_motor_ventana(fallo_en_horizonte) filtrado por NOT NULL, que aísla las filas maduras (etiquetadas) usadas para entrenar.

El índice vectorial (HNSW) sobre intervencion.embedding se crea en la Actividad 9, junto con el cálculo de los embeddings.

7.6 Triggers: detección de eventos y alertas

La base sólo registra hechos; la gestión posterior (agrupar, asignar, escalar, cerrar) vive en la aplicación. Coherente con eso, se implementan dos triggers AFTER INSERT (11_triggers.sql):

1. **medicion → evento**. Al insertar una medición en plata, el trigger compara el valor contra el umbral de la configuración vigente del dispositivo (leído del campo JSONB parametros) y, si lo supera, inserta un evento de tipo umbral superado. Si la lectura viene con mala calidad (fuera de rango o sospechosa), inserta en cambio un evento de dato apuntado al sensor, no al equipo (distingue una falla de instrumentación de una falla real). En ambos casos copia valor_medido y umbral_vigente, para que el evento siga siendo interpretable aunque su medición se purgue al año. El trigger corre sobre plata y no sobre bronce, para que la detección opere siempre sobre datos ya validados.
2. **prediccion → alerta**. Al insertar una predicción en oro, si el score supera el umbral_aplicado, el trigger abre una alerta de origen predictivo. Este trigger vive

en oro y escribe en plata: cruza schemas, algo a considerar en los permisos (Actividad 11).

Las alertas por umbral no las crea un trigger sino la aplicación, agrupando los eventos de un mismo problema (un episodio anómalo genera muchos eventos, pero una sola alerta). En la carga de ejemplo se cargan a mano para representar ese paso.

7.7 Carga de datos de ejemplo

La carga sigue el criterio de la guía de implementación: lo que es catálogo o de bajo volumen se carga a mano para que sea exacto, y las mediciones se generan con un script.

- **Catálogos y activos** (12, 13): las 8 unidades, 8 tipos de variable y 6 tipos de dispositivo; las 12 ubicaciones, 20 dispositivos, 52 sensores y 5 configuraciones. El orden de los INSERT fija los identificadores serial de los que dependen el resto de los datos.
- **Mediciones** (14): 10 días de datos, una lectura cada 15 minutos, sólo para sensores activos de dispositivos operativos (los equipos en mantenimiento o de baja no generan datos). El valor es un valor base por tipo de variable más ruido. El motor 1 recibe además una rampa creciente de vibración que cruza su umbral, lo que hace que el trigger genere eventos reales. Se incluye una lectura de mala calidad para ejercitar el evento de instrumentación.
- **Agregados y features** (15): se calculan con INSERT ... SELECT sobre las mediciones. Al ser datos derivados, si se pierden se recalculan.
- **Trazabilidad y predicciones** (16): modelos, corridas, métricas y predicciones. Las predicciones que superan el umbral disparan las alertas predictivas vía trigger.
- **Gestión** (17): usuarios, las alertas por umbral, y las órdenes de trabajo e intervenciones derivadas.
- **Bronce** (18): una muestra del dato crudo, incluida una fila rechazada por validación (que no se promueve a plata).

Se introdujeron deliberadamente algunos casos de borde —un dispositivo en mantenimiento, uno dado de baja, un sensor fuera de servicio, una configuración historizada con dos versiones— para que las consultas de la Actividad 8 y las reglas del modelo tengan situaciones no triviales que ejercitar.

7.8 Verificación

La reconstrucción completa desde cero produce un estado consistente. Los conteos resultantes de la carga de ejemplo:

Entidad	Filas
medicion	44.207
evento (generados por trigger)	553
alerta por umbral	2
alerta predictiva (generadas por trigger)	2
orden_trabajo	2
intervencion	2
agregado_diario	506
feature_motor_ventana	66
prediccion	4

Los dos triggers quedan verificados por la propia carga: los 553 eventos surgen solos de la rampa de vibración y de la lectura de mala calidad, y de las cuatro predicciones insertadas sólo las dos que superan el umbral de 0,70 generan alerta. Sobre este estado se apoyan las consultas representativas de la Actividad 8.

8. Consultas representativas

8.1 Descripción general

Esta sección presenta seis consultas que ejercitan las capacidades del modelo y responden a necesidades reales del dominio: desde la lectura operativa del estado de un equipo hasta la justificación cuantitativa de la capa analítica. Cada consulta está en su propio archivo .sql dentro de db/consultas/, lista para ejecutarse sobre la base cargada en la Actividad 7.

Todas se probaron sobre la carga de ejemplo (más de un millón de mediciones), verificando que devuelven resultados no triviales.

8.2 Las seis consultas

1. Última lectura de cada sensor de un dispositivo (20_consulta_1_ultima_lectura.sql). Usa DISTINCT ON: por cada sensor del dispositivo conserva la fila de timestamp más reciente. Es la vista operativa típica del estado actual de un equipo.

2. Alertas abiertas por planta, con severidad y antigüedad (21_consulta_2_alertas_abiertas_por_planta.sql). Resuelve la planta de cada dispositivo subiendo la jerarquía de ubicaciones (línea → área → planta) con un CTE recursivo, y lista las alertas no cerradas ordenadas por severidad. Es la consulta sobre la que se verá el efecto de RLS una vez aplicado el aislamiento (Actividad 11): el mismo SELECT, ejecutado por un supervisor, devolverá sólo las alertas de su planta.

3. Promedio y máximo de vibración por dispositivo y día (22_consulta_3_agregacion_vibracion.sql). Agregación clásica sobre un rango temporal, que hace visible la evolución del comportamiento entre motores. En los datos de ejemplo se aprecia la rampa de vibración del motor 1 (de ~3 a ~9,6 mm/s en tres días), mientras el resto se mantiene estable.

4. Media móvil de 7 días (23_consulta_4_media_movil_7d.sql). Función de ventana (AVG() OVER (... ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)) sobre el promedio diario de un sensor. Suaviza el ruido diario y resalta la tendencia, que es lo que alimenta el análisis predictivo. Es el uso de funciones de ventana que pide explícitamente la consigna.

5. Ranking de dispositivos por intervenciones correctivas (24_consulta_5_ranking_intervenciones.sql). Usa una subconsulta correlacionada que, por cada dispositivo, cuenta sus intervenciones sobre órdenes correctivas siguiendo la cadena intervencion → orden_trabajo → alerta → dispositivo. Identifica los equipos más problemáticos.

6. Comparación de rendimiento: plata vs oro (25_consulta_6_explain_plata_vs_oro.sql). Calcula el mismo resultado —el promedio diario de vibración de un sensor— de dos maneras, cada una con EXPLAIN (ANALYZE, BUFFERS): agregando la serie cruda de plata.medicion, o leyendo el agregado ya calculado en oro.agregado_diario. Justifica cuantitativamente por qué existe la capa oro.

8.3 Resultado de la comparación (consulta 6)

Sobre la carga de ejemplo (~1,04 millones de mediciones), las dos formas de obtener el mismo resultado arrojan:

Camino	Filas procesadas	Tiempo de ejecución
Desde plata (agrega la serie cruda)	22.657	~12 ms
Desde oro (lee el agregado precalculado)	81	~0,09 ms

La versión sobre oro es unos dos órdenes de magnitud más rápida: en lugar de recorrer y agrupar decenas de miles de mediciones, lee directamente las filas ya resumidas. La diferencia

crece con el volumen, y a escala real (decenas de millones de filas al año) es lo que hace inviable resolver cada consulta analítica sobre el detalle.

El mismo plan de ejecución muestra el particionado en acción: al consultar por un sensor, PostgreSQL recorre sólo las particiones con datos (junio a agosto) y descarta las vacías al instante. Cuando la consulta además acota por rango de fechas, el particionado permite saltar meses enteros sin leerlos (poda de particiones).

8.4 Nota sobre el volumen de datos

Para que la comparación de la consulta 6 fuera significativa, la carga de ejemplo se amplió respecto de la mínima necesaria para validar el esquema: se generó un backfill histórico de unos tres meses (junio a agosto), con el trigger de detección desactivado durante esa carga masiva —patrón habitual de backfill— y reactivado para la ventana reciente, donde la rampa de vibración del motor 1 dispara los eventos. Con pocas filas, el planificador elige recorrido secuencial en ambos casos y la comparación no mostraría diferencia.

9. Datos semiestructurados, no estructurados y búsqueda vectorial

9.1 Descripción general

La actividad cubre los tres tipos de dato no puramente tabulares del caso: los **semiestructurados** (JSON), los **no estructurados** (texto libre) y la **búsqueda vectorial** sobre ese texto. Los tres se resuelven dentro de PostgreSQL, sin sumar un motor aparte, coherente con la selección tecnológica de la Actividad 6.

9.2 Datos semiestructurados (JSONB)

Lo semiestructurado se maneja con el tipo JSONB: payloads crudos en bronce, umbrales de operación en plata.configuracion_dispositivo.parametros, y criterios/hiperparámetros de entrenamiento en oro. Se elige JSONB sobre JSON porque guarda el documento en forma binaria indexable y permite leer claves internas sin re-parsear el texto. El detalle está en nosql/modelo_nosql.md; la consulta 7 (db/consultas/26_consulta_7_umbrales_jsonb.sql) muestra cómo se extraen los umbrales vigentes de cada dispositivo con jsonb_each y ->>.

9.3 Datos no estructurados y búsqueda vectorial

El dato no estructurado es la observación de texto libre que deja el técnico en cada intervención. El objetivo es poder recuperar intervenciones pasadas con **síntomas parecidos**, descritos en lenguaje natural. La búsqueda por palabras no alcanza: dos textos pueden describir el mismo problema sin compartir vocabulario.

La solución es la **búsqueda vectorial** con la extensión **pgvector**:

- Cada observación se convierte en un *embedding*: un vector que representa su significado. Textos con significado parecido dan vectores cercanos, aunque usen palabras distintas.
- El embedding se guarda en plata.intervencion.embedding, de tipo vector(384). La dimensión 384 corresponde a un modelo liviano multilingüe (all-MiniLM-L6-v2); cambiar de modelo cambia solo esa dimensión.
- La búsqueda es un “vecino más cercano”: se ordena por distancia coseno (operador <=>) y se toman los primeros. Un índice **HNSW** (db/indices_vistas/31_indice_vectorial.sql) lo hace rápido a escala.

Alcance de la implementación

En este trabajo los embeddings se cargan como **vectores de ejemplo** (db/datos/30_embeddings_intervencion) lo que deja el esquema, el índice y la consulta de similitud (db/consultas/32_consulta_8_busqueda_similitud) plenamente operativos. En producción, un servicio externo calcularía el embedding real de cada observación al registrarla; como las observaciones son inmutables, el vector no se desactualiza. El foco del TP es cómo se estructura y se consulta el dato vectorial, no la calidad semántica del modelo de embeddings.

9.4 Riesgo de exposición vía IA

El punto sensible que pide la consigna: la búsqueda por similitud **debe respetar el aislamiento**. No puede devolver una intervención de una planta a la que el usuario no tiene acceso, aunque sea la más parecida. Se garantiza aplicando RLS también sobre intervencion (Actividad 11): el vecino más cercano se busca únicamente entre las filas visibles para el usuario. Sin esa precaución, la capa de IA se convertiría en una vía de fuga de datos entre plantas.

10. Arquitectura de datos

10.1 Descripción general

El sistema se organiza como un **lakehouse lógico dentro de un único PostgreSQL**: la arquitectura Medallion (bronce → plata → oro) implementada con *schemas*, no con sistemas separados. Cada capa representa un grado de madurez del dato, y entre capa y capa hay un proceso que transforma y valida. El dato entra crudo por bronce y va “subiendo” hasta quedar listo para el consumo analítico en oro.

10.2 Diagrama del flujo

El diagrama (fuente editable en docs/arquitectura_datos.mmd) muestra las tres capas, los procesos que llevan de una a otra y los consumidores finales:

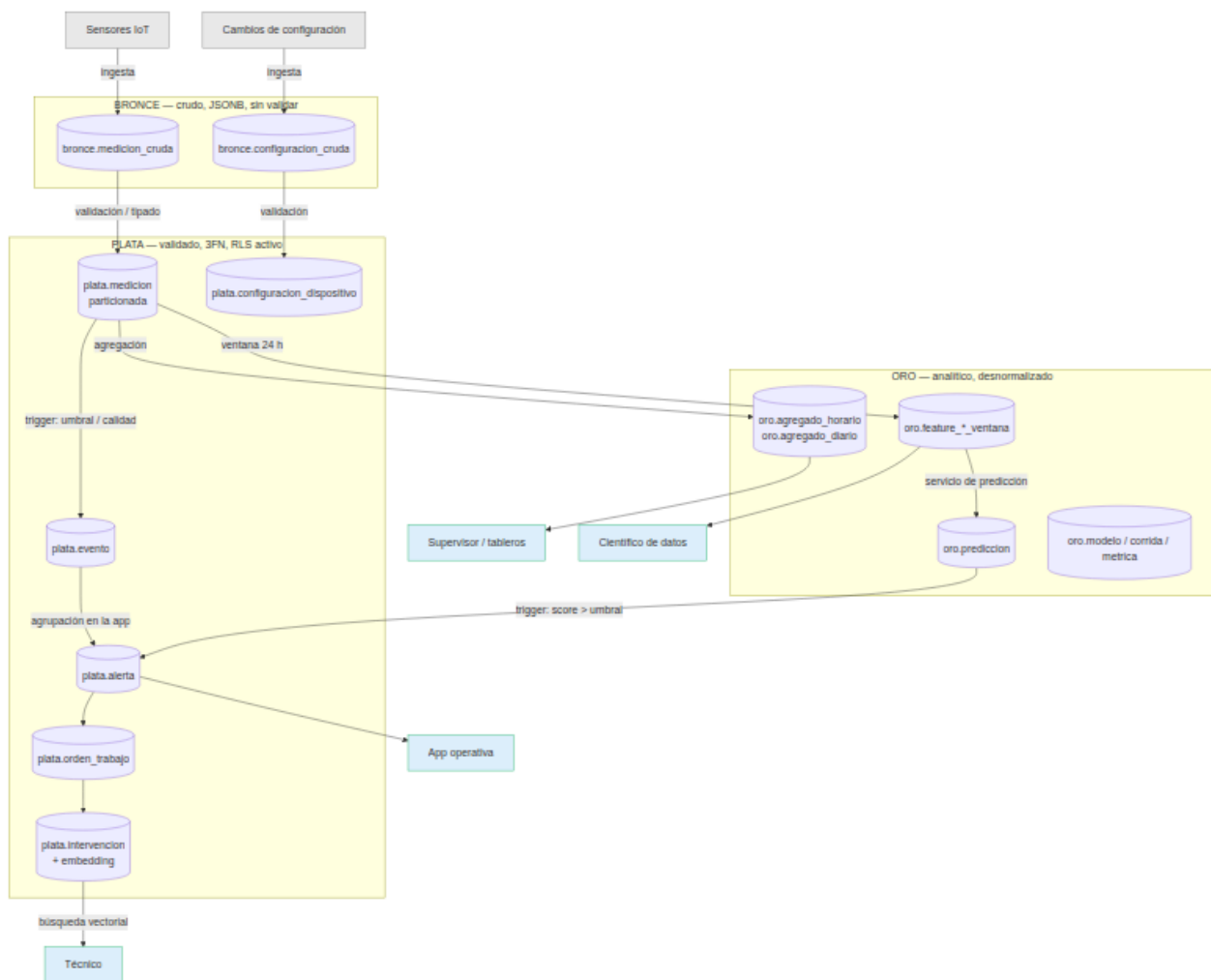


Figura 3: Arquitectura de datos (flujo por capas)

10.3 Las capas y sus procesos

Bronce — crudo. Recibe el dato tal como llega de los sensores y de los cambios de configuración, en JSONB, sin claves foráneas ni validación. Su única función es no perder nada,

aunque el dato venga incompleto o corrupto.

Proceso de validación/tipado (bronce → plata). Convierte el payload crudo en filas tipadas y validadas: descarta o marca lo que no cumple, y promueve el resto a plata. Es el punto donde el dato pasa a ser confiable.

Plata — validado. Capa normalizada (3FN) con integridad referencial y **RLS activo**. Acá vive la operación: mediciones, eventos, alertas, órdenes e intervenciones. Dos procesos internos corren por trigger: una medición fuera de umbral o de mala calidad genera un *evento*, y la agrupación de eventos en *alertas* la hace la aplicación.

Proceso de agregación (plata → oro). Calcula, con `INSERT ... SELECT`, los agregados horarios/diarios y las features por ventana. Es un dato derivado: si se pierde, se recalcula.

Oro — analítico. Capa desnormalizada, optimizada para lectura. Contiene los agregados, las features que consume el modelo, y el bloque de trazabilidad de la predicción (modelo, corrida, métrica, predicción).

Escritura de vuelta. El servicio de predicción lee las features de oro y escribe predicciones en oro; cuando una predicción supera el umbral, un trigger crea una alerta **de vuelta en plata**. Es la única dependencia que va de oro hacia plata.

10.4 Consumidores

- **App operativa** (operarios y técnicos): lee y escribe el estado operativo en plata, siempre bajo RLS.
- **Supervisor:** usa plata para el estado actual y oro para tendencias.
- **Científico de datos:** trabaja sobre oro, sin datos personales, con las features en formato ancho.
- **Técnico:** además consulta intervenciones pasadas por búsqueda vectorial sobre intervencion.

10.5 Límite base / aplicación

Criterio unificador de todo el diseño: **la base registra hechos; la aplicación gobierna procesos con estado**. La base deja constancia de que una alerta se abrió, una orden existe y una intervención se registró (por triggers o inserciones), pero no gestiona las transiciones: agrupar alertas de un mismo problema, asignar técnicos, escalar o cerrar son responsabilidad de la aplicación, fuera del alcance de este trabajo.

11. Seguridad, permisos y aislamiento

11.1 Descripción general

El requisito central es el **aislamiento por planta**: con dos sitios y roles de alcance acotado, un usuario no debe ver datos de una planta que no le corresponde. El aislamiento no queda librado a la aplicación, sino que se resuelve en la propia base con **Row-Level Security (RLS)**. Se suma el ocultamiento de columnas sensibles (la identidad del técnico) para el científico de datos. El DDL está en db/estructura/40_seguridad_rls.sql.

11.2 Doble identidad: roles de base vs usuarios de aplicación

Los usuarios finales (operarios, técnicos, supervisores) **no tienen credencial de base de datos**. La aplicación se conecta con un único **rol de servicio** (app_servicio) y comunica quién es el usuario final por una variable de sesión (app.usuario_id) al inicio de cada transacción (SET LOCAL app.usuario_id = ...). Las políticas RLS leen esa variable con current_setting('app.usuario_id'). Así, un mismo rol de conexión atiende a todos los usuarios, pero cada uno ve solo lo suyo.

Se definen dos roles sin login: app_servicio (la app) y rol_cientifico (acceso analítico restringido).

11.3 Aislamiento por planta con RLS

La planta de una alerta se resuelve subiendo la jerarquía de ubicaciones (línea → área → planta) con funciones auxiliares (fn_planta_de_ubicacion, fn_planta_de_dispositivo). La política compara la planta del dato con la planta del usuario actual (fn_planta_usuario_actual), y deja pasar todo a los usuarios de alcance global —científico y administrador, con id_ubicacion en NULL— vía fn_usuario_es_global.

Se aplicó RLS sobre alerta (el caso testigo) y sobre intervencion (para que la búsqueda vectorial no cruce plantas). Las demás tablas de plata siguen el mismo patrón. La verificación sobre los datos de ejemplo:

Usuario	Alcance	Alertas visibles
Supervisor Planta Norte	su planta	3
Supervisor Planta Sur	su planta	1
Científico de datos	global	4 (todas)

Los 3 + 1 de los supervisores suman exactamente las 4 que ve el usuario global: el aislamiento filtra bien, sin perder ni filtrar filas.

11.4 Ocultamiento de columnas sensibles

RLS es un control a nivel de **fila**: sirve para decidir qué filas ve alguien, pero no para ocultar una **columna** dentro de una fila visible. El científico de datos puede ver las intervenciones de todas las plantas, pero no debe ver *qué técnico* hizo cada una (id_usuario, dato personal innecesario para su análisis). Eso requiere un mecanismo aparte, y hay dos opciones:

- **Vista en oro sin la columna (elegida)**. Se crea oro.v_intervencion, que proyecta todo menos id_usuario, y se le da acceso al rol_cientifico a esa vista en lugar de a la tabla base. Es simple y explícito: el científico literalmente no tiene por dónde acceder a la columna.

- **Permiso a nivel de columna (alternativa).** GRANT SELECT (col1, col2, ...) sobre la tabla, excluyendo id_usuario. Es más granular y evita crear una vista, pero es menos visible y más fácil de romper al agregar columnas. Queda documentado como comentario en el script.

11.5 Notas de implementación

- El trigger que crea alertas predictivas vive en oro y escribe en plata: **cruza schemas**, así que el rol que lo ejecuta necesita permisos en ambos.
- La búsqueda vectorial hereda el RLS de intervencion: el “vecino más cercano” se busca solo entre filas visibles, cerrando el riesgo de exposición vía IA (Actividad 9).
- El costo del recorrido que RLS hace sobre intervencion (varios saltos hasta la planta) es aceptable por el bajo volumen de la tabla; si creciera, se podría materializar la planta como se hizo con id_dispositivo en alerta.

12. Escalabilidad y rendimiento

12.1 El problema de escala

El volumen lo domina la telemetría: 2 plantas × 10 dispositivos × 52 sensores en total, con frecuencias de 10 a 60 s, dan unas 161.000 lecturas por día, **del orden de 59 millones de filas al año** en medicion. El resto de las entidades (eventos, alertas, órdenes, intervenciones) son varios órdenes de magnitud menores. Todo el diseño de rendimiento apunta a que ese volumen de mediciones no degrade el sistema.

12.2 Particionamiento

plata.medicion está **particionada por rango mensual** sobre timestamp_medicion. Dos beneficios concretos:

- **Consultas por tiempo:** una consulta acotada a un rango de fechas lee solo las particiones involucradas (poda de particiones), no la tabla entera. En el plan de la consulta 6 (Actividad 8) se ve cómo los meses sin datos se descartan al instante.
- **Retención:** purgar el detalle vencido es descartar particiones enteras (DROP TABLE de la partición del mes), operación casi instantánea, en lugar de un DELETE masivo que reescribiría índices y generaría trabajo de vacío.

12.3 Índices

- **BRIN** sobre medicion(timestamp_medicion): para datos naturalmente ordenados por tiempo, un BRIN es muchísimo más chico que un B-tree y suficiente para acotar rangos.
- **B-tree** sobre alerta(id_dispositivo, estado) y evento(id_sensor, timestamp_medicion) para las consultas operativas.
- Índice **parcial** sobre las features etiquetadas (fallo_en_horizonte IS NOT NULL), que aísla las filas de entrenamiento.
- **HNSW** sobre intervencion.embedding para la búsqueda vectorial.

12.4 Retención diferenciada

No todo se conserva igual, según volumen y valor:

Entidad	Retención	Motivo
medicion (detalle)	~1 año en plata	Un año captura la estacionalidad del dominio; el detalle completo es inviable de sostener indefinidamente
Agregados de oro	varios años	Bajo volumen; permiten análisis interanual sin la serie completa
Eventos, alertas, órdenes, intervenciones	período extenso / indefinido	Pocos registros y alto valor histórico (insumo de la búsqueda por similitud)

Que evento no tenga FK hacia medicion (y copie valor_medido/umbral_vigente) es lo que permite purgar mediciones sin perder los eventos ni su interpretabilidad.

12.5 Precálculo analítico

Las tablas de oro (agregados y features) son **datos derivados precalculados**: evitan recorrer millones de mediciones en cada consulta analítica. La Actividad 8 lo mide: obtener el promedio diario de un sensor desde el agregado de oro fue unos dos órdenes de magnitud más rápido que calcularlo sobre la serie cruda de plata (~0,09 ms contra ~12 ms sobre ~1 millón de filas), y la diferencia crece con el volumen. Las features se calculan una vez por ventana y alimentan tanto el entrenamiento como la predicción, con ~175.000 filas al año frente a los ~59 millones de mediciones.

12.6 Síntesis

Particionamiento + BRIN para la serie, precálculo en oro para la analítica, índices acordes a cada patrón de consulta y retención diferenciada por entidad. En conjunto, permiten que el sistema sostenga el volumen anual proyectado sin que las consultas se degraden a medida que crece la historia.